# Index and official sources

Find the topic then verify deeper details

**Current foundations**

Variables 4 Types 5-6 Strings 7 Lists 8 Input 9 Operators 10 Conditions 11 Loops 12-13 Functions 14-15

**Future foundations**

Dictionaries 17 Exceptions 18 Files 19 Modules 20 Classes 21 Testing 22 Style 23 Glossary 24

**More practice**

Nested data 25 Comprehensions 26 Planning 27 Worked examples 28-31

**Official Python tutorial**

docs.python.org/3/tutorial/

**Official built-in types**

docs.python.org/3/library/stdtypes.html

**Search formula**

Search Python plus your goal plus the type: Python sort list, Python convert string to int, Python dictionary get default.



# Python Field Guide

A student reference for reading writing and debugging Python

**Use it like a programmer**

Find the idea you need, study the pattern, change one part, and run a small test.

**Current learning comes first**

Pages 3 through 16 cover the concepts used in class so far. Later pages prepare you for future projects.

**You are allowed to look things up**

The goal is not perfect memory. The goal is choosing the right tool and explaining what your code does.

# How to use this guide

A four step lookup routine

### 1 Name the goal

Say what you want the program to do without using code words.

### 2 Find a pattern

Use the index or headings to locate a small example that matches your goal.

### 3 Adapt carefully

Change names and values, but keep punctuation, indentation, and the overall shape.

### 4 Predict and test

Predict the output, run the smallest useful test, and compare the result with your prediction.

### When asking for help

Share what you expected, what happened, the smallest relevant code, and the complete error message.



# Worked function examples

Connect arguments parameters return values and reuse

### Return a calculation

```
def area(width, height):
    return width * height

room_area = area(4, 3)
# width=4, height=3, room_area=12
```

### Use one function in another

```
def double(number):
    return number * 2

def add_doubled(value, total):
    return total + double(value)

answer = add_doubled(3, 10)  # 16
```

### Return early

```
def label(score):
    if score >= 70:
        return "Pass"
    return "Try again"

result = label(82)  # 'Pass'
```

### List parameter

```
def first_item(items):
    if len(items) == 0:
        return None
    return items[0]

first = first_item(["key", "star"])  # 'key'
```

# Worked loop examples

Follow the changing value after each iteration

### Counter

```
count = 0
for name in ["Ana", "Bo", "Cam"]:
    count += 1
# count changes 0 -> 1 -> 2 -> 3
```

### Accumulator

```
total = 0
for value in [4, 6, 2]:
    total += value
# total changes 0 -> 4 -> 10 -> 12
```

### Build a list

```
doubles = []
for number in range(1, 4):
    doubles.append(number * 2)
# doubles becomes [2, 4, 6]
```

### While loop

```
energy = 3
while energy > 0:
    energy -= 1
# energy changes 3 -> 2 -> 1 -> 0, then stops
```



# What we have learned

Current course map

### Data

Variables, strings, numbers, Boolean values, None, lists, and input.

### Decisions

Comparisons, if, elif, else, and Boolean logic.

### Repetition

for loops, range, while loops, counters, and accumulators.

### Organization

Functions, parameters, arguments, return values, and local variables.

### Problem solving

Tracing, reading errors, testing small pieces, and using precise searches.

# Variables and names

Give values useful labels

### Assignment

```
player_name = "Mario"
lives = 3
is_ready = True
```

### Read it right to left

Python evaluates the expression on the right, then stores that value under the name on the left.

### Reassignment

```
coins = 5
coins = coins + 1
# coins is now 6
```

### Good names

Use lowercase words joined by underscores: total_score, student_name, has_key.

### Constants by convention

Names written in ALL_CAPS usually represent values the program does not intend to change.



# Worked decision examples

Trace which branch executes and why

### One comparison

```
coins = 8
if coins >= 10:
    message = "1-UP"
else:
    message = "Keep going"
# message is 'Keep going'
```

### Combined logic

```
has_key = True
door_locked = True
if has_key and door_locked:
    can_enter = True
# can_enter becomes True
```

### Order of branches

```
score = 95
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
# grade is 'A'; Python stops after the first match
```

### Truthiness

```
items = []
if items:
    message = "Items found"
else:
    message = "Empty"
# an empty list acts False, so message is 'Empty'
```

# Worked types examples

Predict the type and value before reading the answer

### Integer and float

```
points = 7
bonus = 2.5
total = points + bonus
# total is 9.5 and its type is float
```

### String versus number

```
text = "12"
number = int(text)
result = number + 3
# result is 15, not '123'
```

### Boolean result

```
temperature = 72
is_warm = temperature >= 70
# is_warm is True and its type is bool
```

### None result

```
def show_score(score):
    print(score)

value = show_score(10)
# value is None because there is no return
```



# Types overview

A type tells Python what a value is and what it can do

### Core types

int whole number float decimal number str text bool True or False NoneType no value

### Collection types

list ordered and changeable tuple ordered and fixed dict key-value pairs set unique values

### Inspect a type

```
score = 12
print(type(score))      # <class 'int'>
print(isinstance(score, int))  # True
```

### Why types matter

The + operator adds numbers but joins strings. Methods also depend on type: lists have append, strings do not.

### Type errors

A TypeError often means an operation received a kind of value it cannot use.

# Numbers booleans and None

Three kinds of values that often control programs

### Numbers

```
whole = 7          # int
decimal = 2.5      # float
result = 7 / 2      # 3.5
```

### Useful operators

+ add - subtract * multiply / divide // floor divide % remainder ** power

### Boolean values

True and False are bool values. Comparisons such as score >= 10 produce a Boolean result.

### None

None means no value. It is not zero, False, or an empty string. Check it with value is None.

### Function connection

A function that reaches its end without return automatically returns None.



# Plan a program

Turn a large idea into testable steps

### Inputs

What information enters the program? What type should each input have?

### Outputs

What should the user see or receive? What should a function return?

### Rules

Write the decisions and repeated actions as plain-language steps before writing Python.

### Functions

Give each major job a name, inputs, and one expected output or effect.

### Tests

Create one normal example, one boundary example, and one invalid example before implementation.

# Comprehensions and sorting

Future shortcuts after loops make sense

### List comprehension

squares = [number ** 2 for number in range(6)]

### Filtered comprehension

passing = [score for score in scores if score >= 70]

### Sorted copy

```
ordered = sorted(scores)
descending = sorted(scores, reverse=True)
```

### Choose clarity

Use a normal loop when the shortcut becomes hard to explain. Shorter code is not always clearer code.



# Strings

Work with text

### Create and combine

```
first = "Ada"
last = 'Lovelace'
full = first + " " + last
```

### Index and slice

```
word = "Python"
word[0]      # 'P'
word[-1]     # 'n'
word[1:4]    # 'yth'
```

### Common methods

name.strip() name.lower() name.upper()
text.replace("old", "new") text.split(",")

### Formatted strings

```
name = "Sam"
score = 92
message = f"{name} scored {score}"
```

### Strings are immutable

String methods create new strings. They do not change the original string in place.

# Lists and tuples

Store ordered collections

### List

```
items = ["key", "star"]
items.append("mushroom")
items[0] = "map"
```

### Read parts

items[0] is the first item. items[-1] is the last. items[1:3] returns a slice.

### Common list tools

len(items) value in items items.remove(value)
items.pop() items.sort() items.reverse()

### Tuple

```
location = (12, 8)
x, y = location
```

### Main difference

Lists are mutable, so their contents can change. Tuples are immutable after creation.



# Nested data

Future structures inside structures

### List of dictionaries

```
students = [
    {"name": "Ava", "score": 88},
    {"name": "Noah", "score": 94},
]
```

### Read a nested value

first_name = students[0]["name"]

### Loop through records

```
for student in students:
    print(student["name"], student["score"])
```

### Work one layer at a time

First identify the outer type, select one value, then apply the operation for the inner type.

# Glossary

Words programmers use

### Expression and statement

An expression produces a value. A statement performs an action such as assignment or return.

### Function method and argument

A function is callable code. A method belongs to an object. An argument is a value passed into a call.

### Mutable and immutable

Mutable objects can change in place. Immutable objects cannot. Lists are mutable; strings and tuples are immutable.

### Iterable and iteration

An iterable can supply values one at a time. One pass through a loop body is an iteration.

### Bug and exception

A bug is incorrect behavior. An exception is a runtime signal that normal execution cannot continue.



# Input and conversion

Turn user text into the type you need

### Input is always text

```
age_text = input("Age: ")
print(type(age_text))  # str
```

### Convert

```
age = int(age_text)
height = float(input("Height: "))
label = str(42)
```

### Conversion can fail

int("12") works. int("twelve") raises ValueError because the text is not numeric.

### Safe beginner pattern

```
text = input("Whole number: ")
if text.isdigit():
    number = int(text)
else:
    print("Use digits only")
```

# Operators and expressions

Build values from other values

### Arithmetic

```
total = price * quantity
average = total / count
remainder = number % 2
```

### Comparison

== equal != not equal < less > greater <= at most >= at least

### Boolean logic

and requires both sides. or requires either side. not reverses the truth value.

### Membership and identity

item in items checks membership. value is None checks whether value is the single None object.

### Order

Parentheses run first, then arithmetic, comparisons, not, and, and finally or. Use parentheses when clarity matters.



# Style and readable code

Make programs easier for people to understand

### Indent consistently

Use four spaces for each block. Do not mix tabs and spaces.

### Name clearly

Prefer calculate_average over calc or x. Boolean names often begin with is, has, or can.

### Keep functions focused

Split a long task into smaller named functions with clear inputs and return values.

### Comments explain why

Do not narrate obvious code. Explain a decision, rule, assumption, or surprising workaround.

### Whitespace helps

Use blank lines between major parts and spaces around operators so code is easier to scan.

# Testing

Future confidence checks

### Arrange act assert

Arrange inputs, act by calling the code, then assert the expected result.

### Simple checks

```
assert power(7, 5) == 12
assert double(0) == 0
assert double(-3) == -6
```

### Test boundaries

Try empty input, zero, one item, negative values, missing values, and the smallest or largest allowed value.

### One behavior per test

A focused test makes failures easier to understand and repair.



# Conditions

Choose which branch runs

### Basic pattern

```
if coins >= 10:
    print("1-UP")
elif coins >= 5:
    print("Almost")
else:
    print("Keep going")
```

### Only one branch

Python checks conditions from top to bottom and runs the first matching branch.

### Truthiness

False, None, zero, and empty strings or collections act false in a condition. Most other values act true.

### Common mistake

Use == to compare two values. Use = to assign a value to a name.

# For loops and range

Repeat across a collection or number sequence

### Loop through values

```
for enemy in ["Boo", "Goomba"]:
    print(enemy)
```

### What range produces

range(4) gives 0, 1, 2, 3. The stop value is not included.

### Range forms

range(4) # start 0, stop 4
range(2, 5) # 2, 3, 4
range(2, 9, 2) # 2, 4, 6, 8

### Accumulator

```
total = 0
for coin in [2, 4, 6]:
    total += coin
```



# Classes and objects

Future custom types

### Class

```
class Player:
    def __init__(self, name):
        self.name = name
        self.lives = 3

    def lose_life(self):
        self.lives -= 1
```

### Object

```
hero = Player("Mario")
hero.lose_life()
print(hero.lives)
```

### Mental model

A class defines a new type. An object is one instance. Attributes store state; methods define behavior.

### self

Inside a method, self refers to the particular object receiving the method call.

# Modules and libraries

Future reusable code

### Import a module

```
import random
choice = random.choice(items)
```

### Import one name

```
from math import sqrt
distance = sqrt(81)
```

### Your own module

Put related functions in helpers.py, then use import helpers in another file.

### Main guard

```
def main():
    print("Program starts")

if __name__ == "__main__":
    main()
```



# While loops

Repeat while a condition remains true

### Basic pattern

```
lives = 3
while lives > 0:
    print(lives)
    lives -= 1
```

### Two required ideas

Identify the condition that continues the loop and the update that eventually makes it false.

### Infinite loop warning

A while loop continues forever if its condition never becomes false.

### Loop controls

break exits the nearest loop. continue skips the rest of the current iteration.

### Choose wisely

Use for for a collection or known sequence. Use while when stopping depends on changing state.

# Functions

Package a task under a name

### Define and call

```
def power(base, bonus):
    total = base + bonus
    return total

score = power(7, 5)
```

### Vocabulary

Parameters are names in the definition. Arguments are values supplied by the call.

### Return

return ends the function and sends a value to the caller. Save or use that returned value.

### Print is different

print displays information. return produces a value that the rest of the program can reuse.



# Files and JSON

Future persistent data

### Read text

```
with open("notes.txt", "r", encoding="utf-8") as file:
    text = file.read()
```

### Write text

```
with open("score.txt", "w", encoding="utf-8") as file:
    file.write(str(score))
```

### JSON data

```
import json
with open("player.json", "r") as file:
    player = json.load(file)
```

### Why with matters

The with statement closes the file automatically, even if an error interrupts the block.

# Exceptions

Future error handling

### Expected conversion problem

```
try:
    age = int(input("Age: "))
except ValueError:
    print("Enter a whole number")
```

### Common exceptions

SyntaxError invalid code shape NameError unknown name TypeError wrong kind of value IndexError missing position

### Catch narrowly

Catch only errors you expect and can handle. Do not use a broad except to hide unexpected failures.

### else and finally

An else block runs if no exception occurred. A finally block runs whether an exception occurred or not.



# Functions in depth

Understand scope defaults and None

### Local scope

```
def double(number):
    result = number * 2
    return result
# result does not exist here
```

### Default argument

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}"
```

### Keyword argument

message = greet(name="Maya", greeting="Hi")

### None result

```
def announce(message):
    print(message)

result = announce("Ready")  # None
```

### Function habit

Prefer small functions that do one clear job and have names describing that job.

# Debugging current work

A repeatable process for getting unstuck

### 1 Read the final error line

Identify the error type and message before changing anything.

### 2 Inspect the named line

Check spelling, punctuation, indentation, and the types of the values involved.

### 3 Trace state

Use print or a debugger to observe values before the failure and during each loop iteration.

### 4 Reduce the problem

Test one expression or function call. Remove unrelated code until the problem is easy to see.

### 5 Fix and retest

Change one cause at a time, rerun the same test, and add a test that prevents the mistake from returning.



# Dictionaries and sets

Future collection tools

### Dictionary

```
player = {"name": "Luigi", "lives": 3}
print(player["name"])
player["lives"] -= 1
```

### Safe dictionary lookup

power = player.get("power", "small")

### Loop through pairs

```
for key, value in player.items():
    print(key, value)
```

### Set

```
tags = {"python", "game"}
tags.add("loops")
```

### When to choose each

Use a dictionary for labeled fields. Use a set for unique values and fast membership checks.